

Program	Bachelor of Technology (B.Tech.)	Semester - 5
Type of Course	Core Courses	
Prerequisite	Basic understanding of programming concepts such as variables, loops, and conditionals—ideally covered in the Programming for Problem Solving course from an earlier semester. Familiarity with using the command-line interface (CLI) is recommended. No prior Django experience is required	
Rationale	To provide a strong foundation in Python programming with OOP concepts. To develop dynamic, secure, and scalable web applications using Django. To integrate frontend technologies with Django templates. To understand database handling using Django ORM. To deploy Django applications to real-world environments.	
Effective From A.Y.	2024-25	

Teaching Scheme (Contact Hours)				Examination Scheme				
Lecture	Tutorial	Lab	Credit	Theory Marks		Practical Marks		Total Marks
				T	T	P	P	
3	-	2	4	70	30	-	50	150

SEE - Semester End Examination, T - Internal Theory, P - Internal Practical

Course Content		T - Teaching Hours W - Weightage	
Sr.	Topics	T	W
1	Python Fundamentals for Web Development Python syntax, variables, and data types, Conditional statements and loops, Functions, modules, and packages, Exception handling and file I/O, Object-Oriented Programming in Python.	8	20
2	Introduction to Web Development & Django Basics: Introduction to web development (HTTP, URL, client-server), Django installation and project structure, Django views, URLs, templates, Django development server and settings	10	20
3	Django Models, ORM and Databases: Django Models and migrations, QuerySets and ORM methods, Model relationships (OneToOne, ForeignKey, ManyToMany), Admin interface customization, Database Operations	12	20
4	Advanced Django Features & Forms: Django forms: built-in and ModelForms, Form validation and error handling, User authentication: login, logout, registration, Middleware, sessions, and cookies, Class-based views	10	20
5	REST APIs with Django REST Framework & Deployment Basics: Django REST Framework introduction, Building simple APIs (GET, POST), Consuming APIs with Django, Deployment using Heroku, Render, or PythonAnywhere, Final project planning and implementation.	15	20
Total		55	100

Suggested Distribution Of Theory Marks Using Bloom's Taxonomy					
Level	Remembrance	Understanding	Application	Analyze	Create
Weightage	20	25	20	15	20

NOTE : This specification table shall be treated as a general guideline for the students and the teachers. The actual distribution of marks in the question paper may vary slightly from above table.

Course Outcomes

At the end of this course, students will be able to:

CO1	Understand and apply core Python programming concepts including OOP, file handling, and modules.
CO2	Design dynamic and structured web pages using Django's view, URL routing, and template system.
CO3	Develop and manage databases using Django models, relationships, and ORM queries effectively.
CO4	Implement secure web features such as authentication, form validation, and middleware logic.
CO5	Create and deploy full-stack Django applications integrated with RESTful APIs and cloud hosting.

CO PO Mapping

CO	CO - 1	CO - 2	CO - 3	CO - 4	CO - 5
PO - 1	3	2	3	2	3
PO - 2	2	2	3	3	3
PO - 3	2	3	2	3	3
PO - 4	1	1	2	2	2
PO - 5	2	3	3	3	3
PO - 6	1	2	1	2	2
PO - 7	1	1	1	1	2
PO - 8	1	1	1	2	1
PO - 9	2	2	2	2	3
PO - 10	1	2	1	1	2
PO - 11	1	1	1	2	3
PO - 12	2	2	2	2	3

Reference Books

1.	Two Scoops of Django 3.x: Best Practices for the Django Web Framework
----	---

List of Practical

1.	Write a Python program to check if a string is a palindrome (reads the same forwards and backwards).
2.	Write a Python program to manage a library inventory system using classes and objects.
3.	Write a Python program to print the Fibonacci series up to n terms.
4.	Write a Python program to count the number of times each word appears in a string.
5.	Write a Python class for a shopping cart with methods to add, remove, and calculate total price of items.
6.	Write an API endpoint that returns a "404 Not Found" error for invalid routes.
7.	Create a new Django project named MyWebsite and run the development server
8.	Create a Django app named blog and register it in the project's settings..
9.	Create a view that returns a simple "Hello, Django!" response and map it to a URL.
10.	Create an HTML page using Django templates that displays your name and profession.
11.	Create a base template and extend it with two different pages: home.html and about.html.
12.	Define a Book model with fields: title, author, and published_date. Run migrations.
13.	Add a few books via admin and display all book titles on a web page using Django ORM. Create a form where users can submit their name and email. Show submitted values on the next page.
14.	Add login/logout functionality to your Django site using Django built-in auth system. Add a CSS file to style your site and link it in templates using { %static% } tag.
15.	Build a REST API using Django REST Framework to perform CRUD operations on a product model.